

Going to use User model in coming examples:

```
static class User {
    String name;
    Optional<String> email;

    User(String name, Optional<String> email) {
        this.name = name;
        this.email = email;
    }

    Optional<String> getEmail() {
        return email;
    }

    @Override
    public String toString() {
        return name + " (" + email.orElse("no-email") + ")";
    }
}
```

Flatten nested Optionals:

```
static void scenario1_flattenNestedOptionals() {
    System.out.println("\nScenario1: Flatten Nested Optionals");
    Optional<Optional<String>> nested = Optional.of(Optional.of("Hello"));
    Optional<String> flat = nested.flatMap(o -> o.map(String::toUpperCase));
    System.out.println("Result: " + flat); // Optional[HELLO]
}
```

Optional inside Stream:

```
static void scenario2_optionalInStream() {
    System.out.println("\nScenario2: Optional inside Stream");
    List<Optional<String>> optionalList = Arrays.asList(Optional.of("Java"),
Optional.empty(), Optional.of("Stream"));
    List<String> flat = optionalList.stream()
        .flatMap(Optional::stream)
        .collect(Collectors.toList());
    System.out.println("Flat values: " + flat);
}
```

Optional chaining with possible null objects:

```
static void scenario3_optionalChainingWithNull() {
    System.out.println("\nScenario3: Optional chaining with possible null");
}
```

```

List<User> users = Arrays.asList(
    new User("Alice", Optional.of("alice@example.com")),
    null,
    new User("Bob", Optional.empty())
);

List<String> emails = users.stream()
    .map(u -> Optional.ofNullable(u).flatMap(User::getEmail))
    .flatMap(Optional::stream)
    .toList();
System.out.println("Emails: " + emails);
}

```

Optional with max/min:

```

static void scenario4_optionalMaxMin() {
    System.out.println("\nScenario4: Optional with max/min");
    List<Integer> numbers = List.of();
    Optional<Integer> max = numbers.stream().max(Integer::compare);
    System.out.println("Max or default (-1): " + max.orElse(-1));
}

```

GroupingBy with Optional values:

```

static void scenario5_groupingWithOptional() {
    System.out.println("\nScenario5: GroupingBy with Optional");
    List<User> users = Arrays.asList(
        new User("Alice", Optional.of("alice@example.com")),
        new User("Bob", Optional.empty())
    );

    Map<String, List<User>> grouped = users.stream()
        .collect(Collectors.groupingBy(u -> u.getEmail().orElse("unknown")));
    System.out.println(grouped);
}

```

findFirst vs findAny in parallel stream:

```

static void scenario6_findFirstVsFindAny() {
    System.out.println("\nScenario6: findFirst vs findAny");
    List<String> data = Arrays.asList("A", "B", "C", "D");
    Optional<String> first = data.parallelStream().findFirst();
    Optional<String> any = data.parallelStream().findAny();
    System.out.println("findFirst: " + first + ", findAny: " + any);
}

```

Filter out empty Optionals:

```
static void scenario7_filterOptionalEmptyValues() {
    System.out.println("\nScenario7: Filter out empty Optionals");
    List<Optional<String>> list = Arrays.asList(Optional.empty(), Optional.of("X"),
Optional.of("Y"));
    List<String> present = list.stream()
        .filter(Optional::isPresent)
        .map(Optional::get)
        .toList();
    System.out.println("Present values: " + present);
}
```

Flatten nested List<Optional<List<String>>>:

```
static void scenario8_flatMapNestedLists() {
    System.out.println("\nScenario8: Flatten nested List<Optional<List<String>>>");
    List<Optional<List<String>>> nested = Arrays.asList(
        Optional.of(Arrays.asList("a", "b")),
        Optional.empty(),
        Optional.of(Arrays.asList("c", "d"))
    );
    List<String> flat = nested.stream()
        .flatMap(Optional::stream)
        .flatMap(List::stream)
        .toList();
    System.out.println("Flattened list: " + flat);
}
```

orElse vs orElseGet performance pitfall:

```
static void scenario9_orElseVsOrElseGet() {
    System.out.println("\nScenario9: orElse vs orElseGet");
    Optional<String> empty = Optional.empty();

    String result1 = empty.orElse(expensiveOperation());
    System.out.println("orElse result: " + result1);

    String result2 = empty.orElseGet(StreamOptionalTrickyScenarios::expensiveOperation);
    System.out.println("orElseGet result: " + result2);
}

static String expensiveOperation() {
    System.out.println("Executing expensive operation...");
}
```

```
    return "Default";  
}
```

Short-circuit Optional with Stream.anyMatch:

```
static void scenario10_optionalShortCircuit() {  
    System.out.println("\nScenario10: Short-circuit with Optional and Stream.anyMatch");  
    List<Optional<Integer>> numbers = Arrays.asList(Optional.of(1), Optional.of(2),  
Optional.empty(), Optional.of(4));  
    boolean hasEven = numbers.stream()  
        .flatMap(Optional::stream)  
        .anyMatch(n -> n % 2 == 0);  
    System.out.println("Contains even number? " + hasEven);  
}
```

OptionalFlatMap1: Flatten Optional of List:

```
static void optionalFlatMap1() {  
    System.out.println("\nOptionalFlatMap1: Flatten Optional of List");  
    Optional<List<Integer>> optList = Optional.of(Arrays.asList(1,2,3,4,5));  
    long start = System.nanoTime();  
    List<Integer> flat =  
optList.map(List::stream).orElse(Stream.empty()).collect(Collectors.toList());  
    long end = System.nanoTime();  
    System.out.println("Flattened Optional: " + flat + ", Time(ms): " + (end-  
start)/1_000_000);  
}
```

OptionalFlatMap2: Nested Optionals flatten:

```
static void optionalFlatMap2() {  
    System.out.println("\nOptionalFlatMap2: Nested Optionals flatten");  
    Optional<Optional<String>> nested = Optional.of(Optional.of("hello"));  
    long start = System.nanoTime();  
    Optional<String> flat = nested.flatMap(o->o);  
    long end = System.nanoTime();  
    System.out.println("Flattened nested Optional: " + flat.get() + ", Time(ms): " +  
(end-start)/1_000_000);  
}
```

OptionalFlatMap3: Optional empty handling:

```
static void optionalFlatMap3() {  
    System.out.println("\nOptionalFlatMap3: Optional empty handling");  
    Optional<String> empty = Optional.empty();  
    long start = System.nanoTime();
```

```

String result = empty.flatMap(s->Optional.of(s.toUpperCase())).orElse("default");
long end = System.nanoTime();
System.out.println("Result for empty Optional: " + result + ", Time(ms): " + (end-
start)/1_000_000);
}

```

OptionalFlatMap4: Optional map vs flatMap with list:

```

static void optionalFlatMap4() {
    System.out.println("\nOptionalFlatMap4: Optional map vs flatMap with list");
    Optional<List<String>> opt = Optional.of(Arrays.asList("a","b"));
    long start = System.nanoTime();
    List<String> resMap = opt.map(l->l).orElse(Collections.emptyList());
    List<String> resFlatMap = opt.flatMap(l-
>Optional.of(l)).orElse(Collections.emptyList());
    long end = System.nanoTime();
    System.out.println("Map: " + resMap + ", FlatMap: " + resFlatMap + ", Time(ms): " +
(end-start)/1_000_000);
}

```

OptionalFlatMap5: Optional of Optional empty:

```

static void optionalFlatMap5() {
    System.out.println("\nOptionalFlatMap5: Optional of Optional empty");
    Optional<Optional<String>> opt = Optional.of(Optional.empty());
    long start = System.nanoTime();
    Optional<String> flat = opt.flatMap(o->o);
    long end = System.nanoTime();
    System.out.println("Flattened Optional: " + flat.orElse("empty") + ", Time(ms): " +
(end-start)/1_000_000);
}

```

OptionalFlatMap6: Optional map chaining:

```

static void optionalFlatMap6() {
    System.out.println("\nOptionalFlatMap6: Optional map chaining");
    Optional<String> opt = Optional.of("java");
    long start = System.nanoTime();
    Optional<Integer> len = opt.map(String::toUpperCase).map(String::length);
    long end = System.nanoTime();
    System.out.println("Length after map chaining: " + len.get() + ", Time(ms): " + (end-
start)/1_000_000);
}

```

OptionalFlatMap7: Optional filter + flatMap:

```

static void optionalFlatMap7() {
    System.out.println("\nOptionalFlatMap7: Optional filter + flatMap");
    Optional<String> opt = Optional.of("hello");
    long start = System.nanoTime();
    Optional<String> res = opt.filter(s->s.startsWith("h")).flatMap(s->Optional.of(s.toUpperCase()));
    long end = System.nanoTime();
    System.out.println("Filtered + flatMap: " + res.get() + ", Time(ms): " + (end-start)/1_000_000);
}

```

OptionalFlatMap8: Optional stream flatten:

```

static void optionalFlatMap8() {
    System.out.println("\nOptionalFlatMap8: Optional stream flatten");
    Optional<List<Integer>> optList = Optional.of(Arrays.asList(1,2,3));
    long start = System.nanoTime();
    List<Integer> flat =
optList.map(List::stream).orElse(Stream.empty()).collect(Collectors.toList());
    long end = System.nanoTime();
    System.out.println("Flattened Optional stream: " + flat + ", Time(ms): " + (end-start)/1_000_000);
}

```